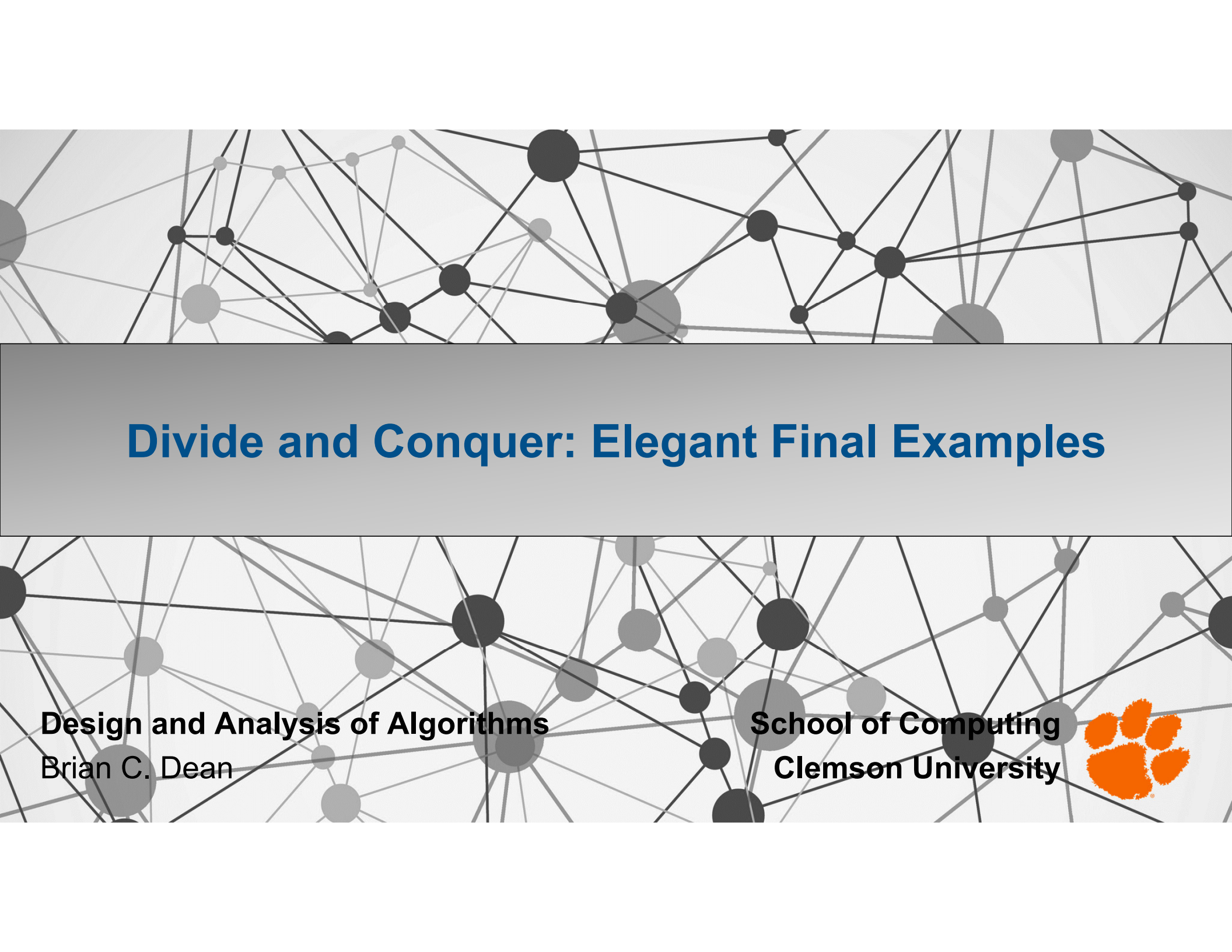




Divide and Conquer: Elegant Final Examples

Design and Analysis of Algorithms

Brian C. Dean

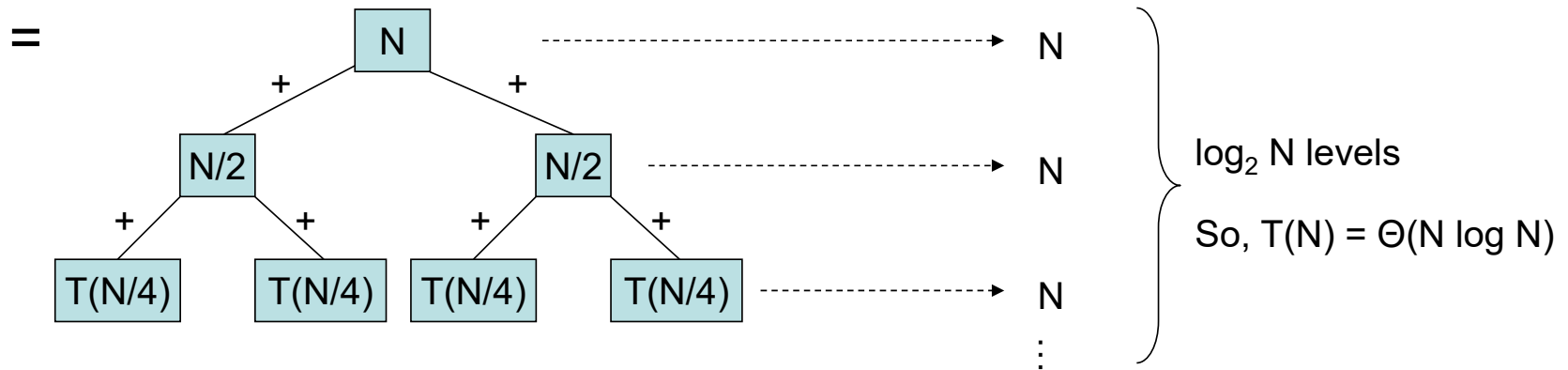
School of Computing

Clemson University

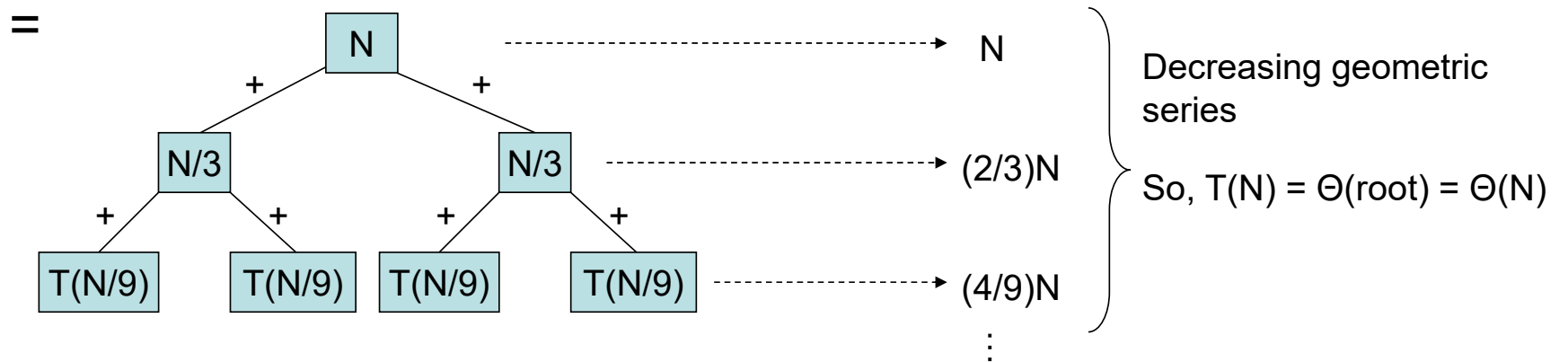


Recall: Tree Expansions

- $T(N) = 2T(N/2) + N$

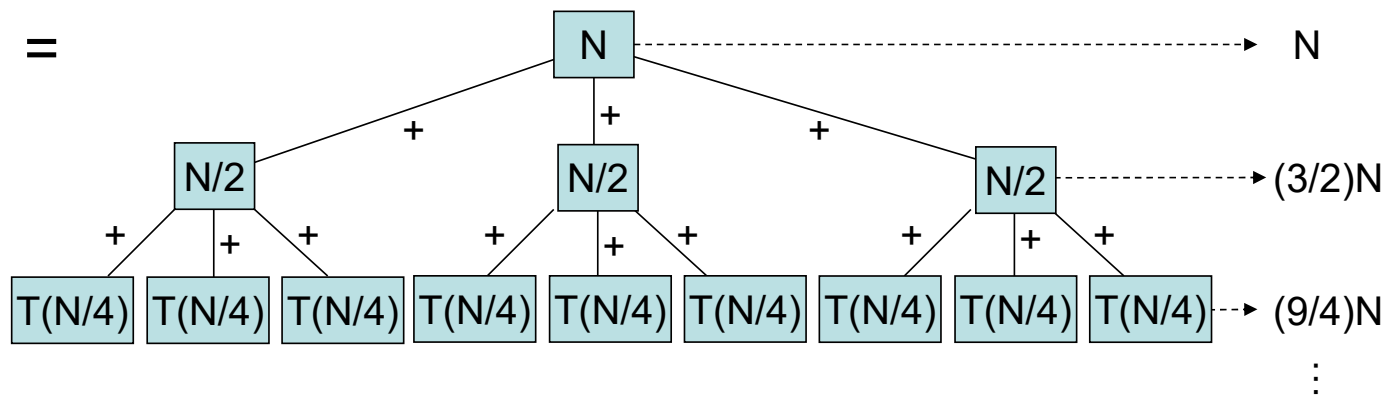


- $T(N) = 2T(N/3) + N$



Recall: Tree Expansions

- $T(N) = N + 3T(N/2)$



Increasing series: $T(N) = \Theta(\text{leaf contribution})$
 $\Theta(n^{\log_2 3})$

Extra Log Factors...

- E.g., $T(N) = 2T(N/2) + \Theta(N \log N)$
- To solve $T(N) = aT(N/b) + N^\alpha \log^\beta N$:
 - Decreasing series: $T(N) = \Theta(\text{root}) = \Theta(N^\alpha \log^\beta N)$
 - Increasing series: $T(N) = \Theta(\text{leaves}) = \Theta(N^{\log_b a})$
 - Unchanging series: $T(N) = \Theta(N^\alpha \log^{\beta+1} N)$
- Approach for solving:
 - Temporarily pretend “ $\log^\beta N$ ” term not there.
 - Solve recurrence.
 - Adjust solution appropriately as above.

Practice

$$T(n) = 2T(n/2) + \Theta(n \log n)$$

$$T(n) = 2T(n/2) + \Theta(n \log^2 n)$$

$$T(n) = 2T(n/2) + n \log^2 n - 17n + 5$$

$$T(n) = 3T(n/2) + \Theta(n \log^{10} n)$$

$$T(n) = 2T(n/3) + \Theta(n \log^{10} n)$$

$$T(n) = T(n/2) + \Theta(\log n)$$

Practice

$$T(n) = 2T(n/2) + \Theta(n \log n) \quad \Theta(n \log^2 n)$$

$$T(n) = 2T(n/2) + \Theta(n \log^2 n) \quad \Theta(n \log^3 n)$$

$$T(n) = 2T(n/2) + n \log^2 n - 17n + 5 \quad \Theta(n \log^3 n)$$

$$T(n) = 3T(n/2) + \Theta(n \log^{10} n) \quad \Theta(n^{\log_2 3})$$

$$T(n) = 2T(n/3) + \Theta(n \log^{10} n) \quad \Theta(n \log^{10} n)$$

$$T(n) = T(n/2) + \Theta(\log n) \quad \Theta(\log^2 n)$$

More Sophisticated Example: Stable In-Place Sorting

Algorithm	Runtime	Stable	In-Place?
Bubble Sort	$O(n^2)$	Yes	Yes
Insertion Sort	$O(n^2)$	Yes	Yes
Merge Sort	$\Theta(n \log n)$	Yes	No
Randomized Quicksort	$\Theta(n \log n)$ w/high prob.	No*	Yes*
Deterministic Quicksort	$\Theta(n \log n)$	No*	Yes*
Heap Sort	$\Theta(n \log n)$	No*	Yes*

- Is stable in-place sorting possible in $O(n \log n)$ time? Yes, but very complicated.
- However, there are simple $O(n \log^2 n)$ time approaches...

* = can be transformed into a stable, out-of-place algorithm

More Sophisticated Example: Stable In-Place Sorting

Algorithm	Runtime	Stable	In-Place?
Bubble Sort	$O(n^2)$	Yes	Yes
Insertion Sort	$O(n^2)$	Yes	Yes

Example #1 of utility of stable sorting:

Sort your inbox on “date received”, then stably by “from address”.
Items received from same person remain sorted by date.

Example #2:

Radix sort: sorts n integers of size at most n^{constant} in $O(n)$ time.

-
- However, there are simple $O(n \log^2 n)$ time approaches...

* = can be transformed into a stable, out-of-place algorithm

Recall: Counting Sort

- Sorts n integers of size $< C$ in $O(n + C)$ time.
- If $C = O(n)$, this is $O(n)$ time.
- With care, can be made **stable**.

$A[1..n]$:

3
4
1
3
2
2
3
3
1

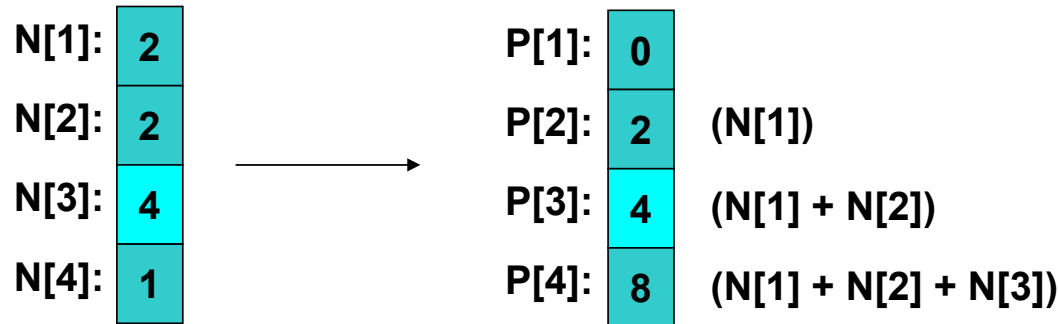
$N[1]:$	2
$N[2]:$	2
$N[3]:$	4
$N[4]:$	1



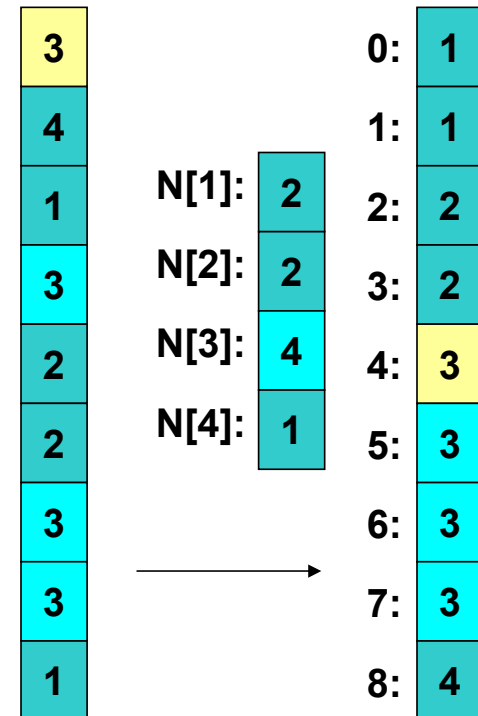
1
1
2
2
3
3
3
3
4

Recall: Counting Sort

- Sorts n integers of size $< C$ in $O(n + C)$ time.
- If $C = O(n)$, this is $O(n)$ time.
- With care, can be made **stable**.



$A[1..n]:$



Radix Sort

- Write integers in some base (radix), r .
(usually we set $r = n$)
- Sort on each digit, starting with the **least** significant, using a **stable** sort
(usually counting sort)
- # digits = $\log_n C$, a constant if $C = n^{O(1)}$.
- Runtime $O(n)$ if $C = n^{O(1)}$.
- Dates back to mechanical sorting machines for punched cards!

A[1...n]:

4 4 6	1 2 0	3 0 9	1 1 5
7 1 2	4 2 0	5 0 9	1 2 0
3 0 9	7 1 2	7 1 2	1 4 6
4 4 2	4 4 2	7 1 5	3 0 9
4 3 5	8 9 2	1 1 5	4 2 0
1 2 0	4 3 5	1 2 0	4 3 5
6 3 8	7 1 5	4 2 0	4 3 7
7 1 5	1 1 5	4 3 5	4 4 2
4 3 7	5 9 5	4 3 7	4 4 6
8 9 2	4 4 6	6 3 8	5 0 9
1 1 5	1 4 6	4 4 2	5 9 5
5 0 9	4 3 7	4 4 6	6 3 8
4 2 0	6 3 8	1 4 6	7 1 2
1 4 6	3 0 9	8 9 2	7 1 5
5 9 5	5 0 9	5 9 5	8 9 2

Radix Sort: Forward Direction

- Divide and conquer!
- Multi-way partition on leading digit.
- Then recursively sort groups.
(very much like with quicksort)
- # digits (recursion depth) = $\log_n C$, a constant if $C = n^{O(1)}$.
- $O(n)$ time spent per recursion level.
- Runtime still $O(n)$ if $C = n^{O(1)}$.

4 4 6	→	1 2 0
7 1 2		1 1 5
3 0 9		1 4 6
4 4 2		3 0 9
4 3 5		4 3 5
1 2 0		4 2 0
6 3 8		4 4 2
7 1 5		4 4 6
4 3 7		4 3 7
8 9 2		5 9 5
1 1 5		5 0 9
5 0 9		6 3 8
4 2 0		7 1 5
1 4 6		7 1 2
5 9 5		8 9 2

Warm-Up: In-Place Block Swap

- Suppose we have a length- n array comprised of two blocks A and B of potentially different size:



- How can we rearrange this array **in-place** in $\Theta(n)$ time so that its contents are now:

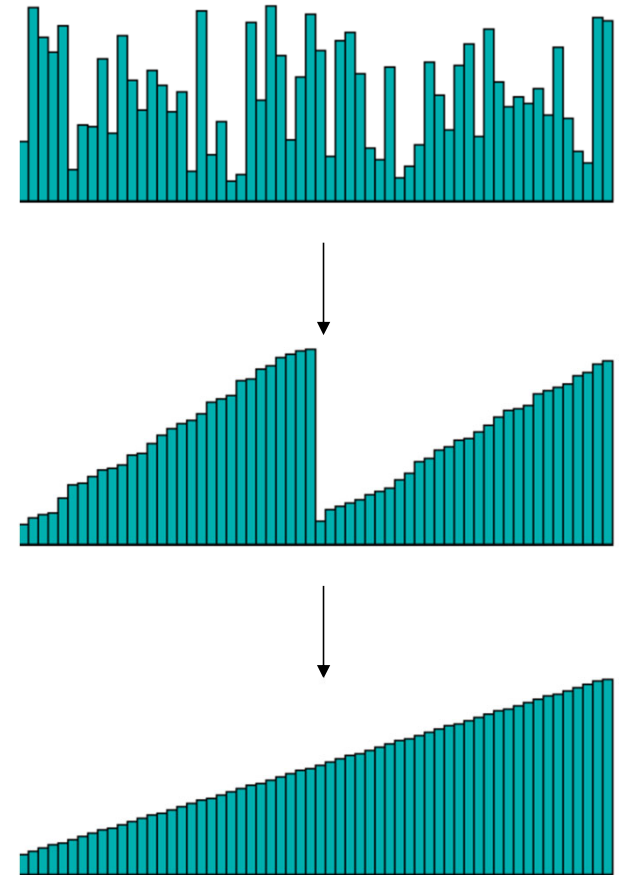


- Note: this is easy if A and B have the same size...

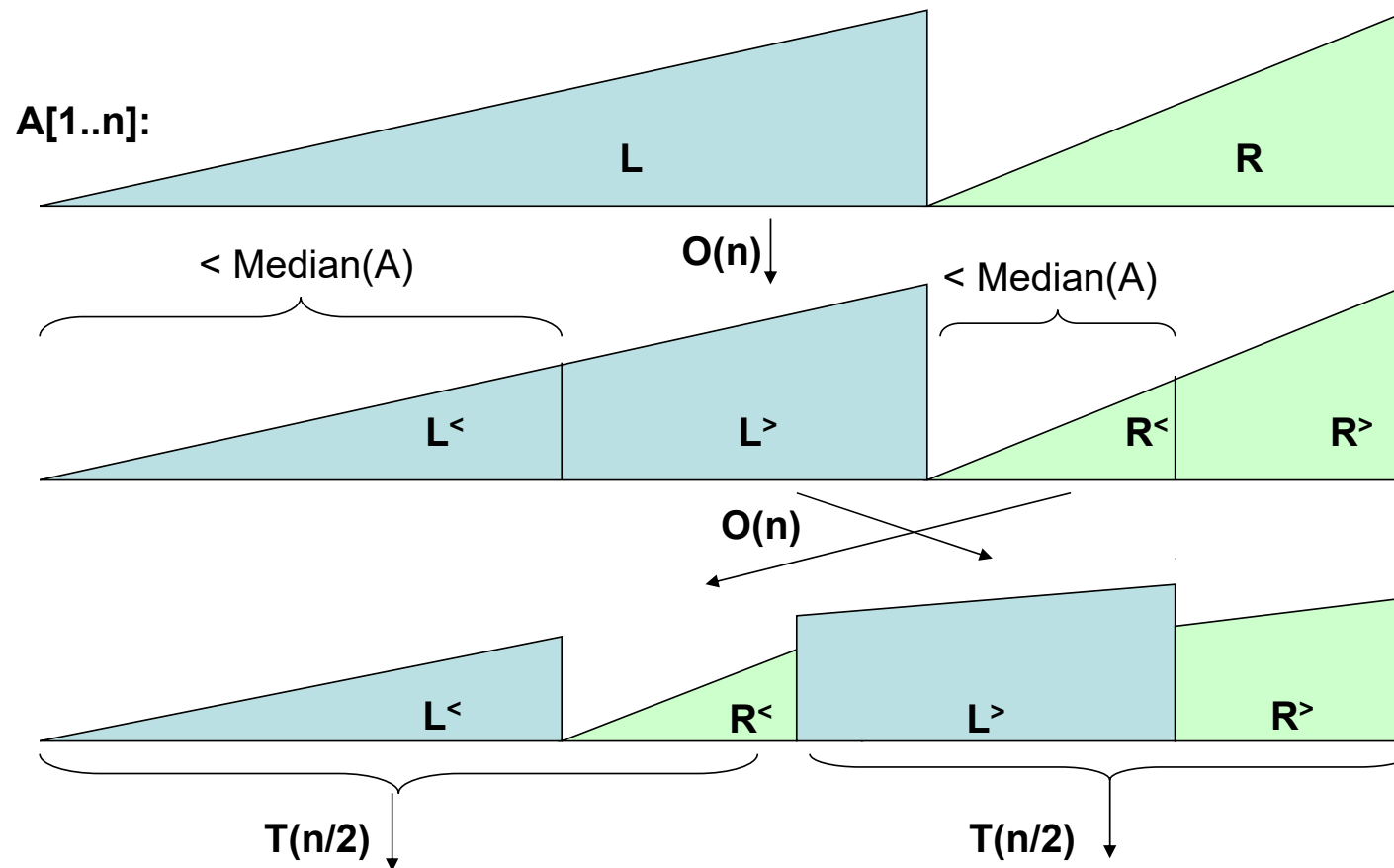
Stable In-Place Merge Sort

- The standard $\Theta(n)$ merging algorithm is stable but not in-place.
- We'll show how to merge in $\Theta(n \log n)$ time in a stable and in-place fashion.
- When used with merge sort, this gives a $\Theta(n \log^2 n)$ runtime:

$$T(n) = 2T(n/2) + \Theta(n \log n)$$

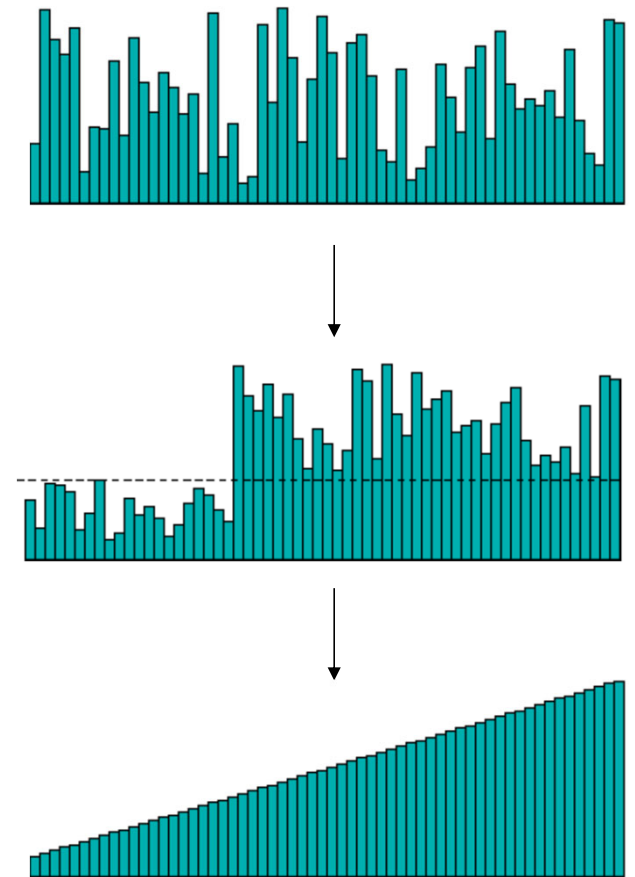


$\Theta(n \log n)$ Stable In-Place Merge

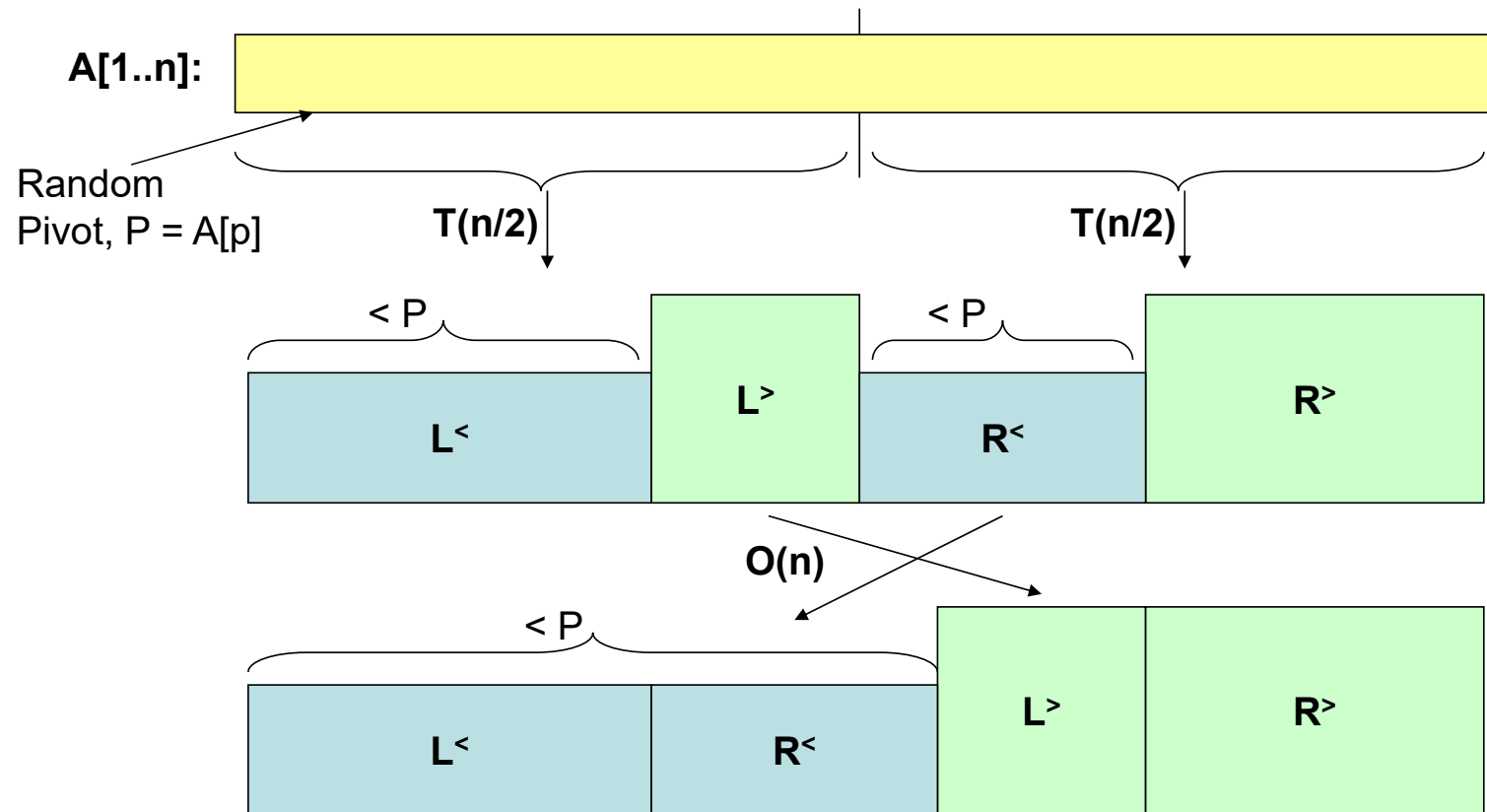


Stable In-Place Quicksort

- The standard $\Theta(n)$ partition algorithm is in-place but not stable.
- We'll show how to partition in $\Theta(n \log n)$ time in a stable and in-place fashion.
- When used with randomized quicksort, this gives a $\Theta(n \log^2 n)$ runtime with high probability.

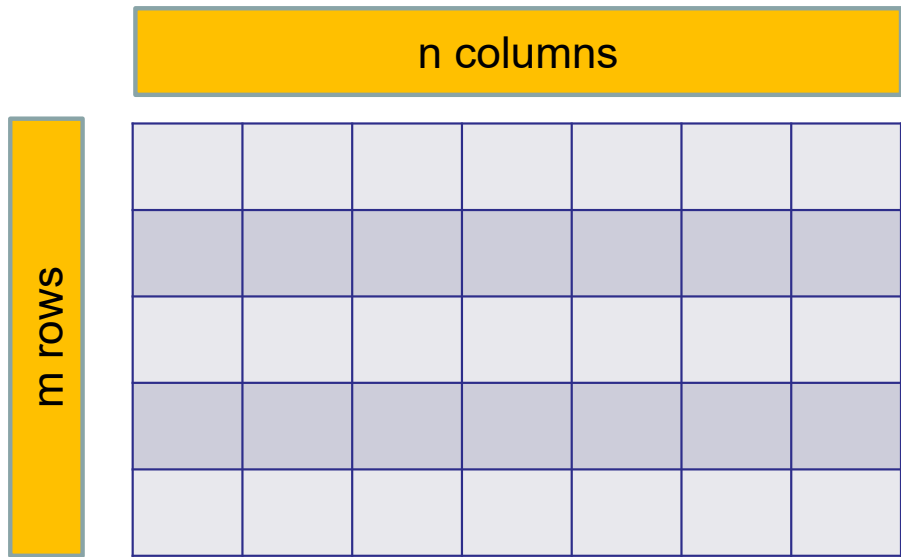


$\Theta(n \log n)$ Stable In-Place Partition

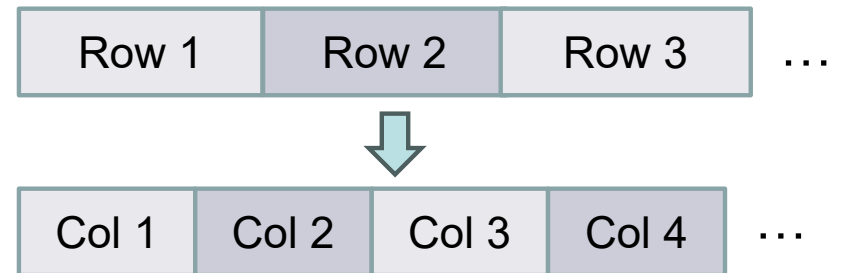


Remember This Problem?

Transpose a rectangular matrix of size $N = mn$ “in place”.



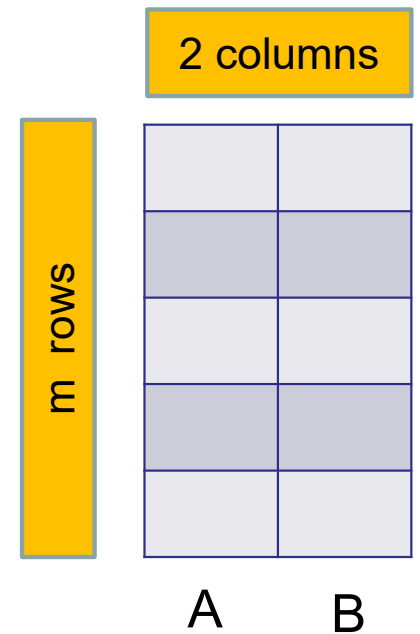
In memory:



We showed how to solve this in $O(N \log N)$ time earlier.

Special Case: Finding the Transpose of an $m \times 2$ Matrix

- $A[1..m]$ and $B[1..m]$: data in the two columns.
- In memory, we start with the matrix stored as:
 $A[1] \ B[1] \ A[2] \ B[2] \ A[3] \ B[3] \ \dots \ A[m] \ B[m]$
- We would like to rearrange it so it's stored as:
 $A[1] \ A[2] \ A[3] \ \dots \ A[m] \ B[1] \ B[2] \ B[3] \ \dots \ B[m]$



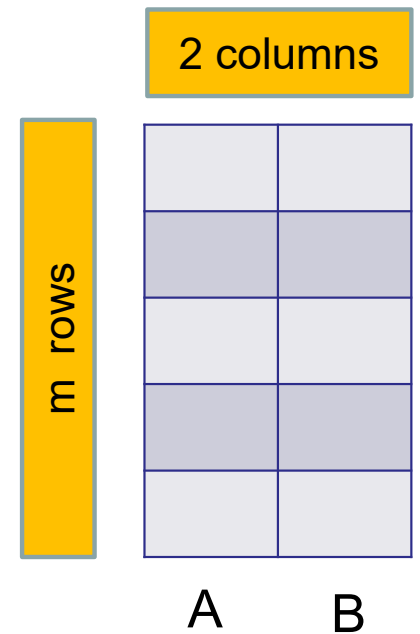
An Inverse Perfect Shuffle...

- $A[1..m]$ and $B[1..m]$: data in the two columns.

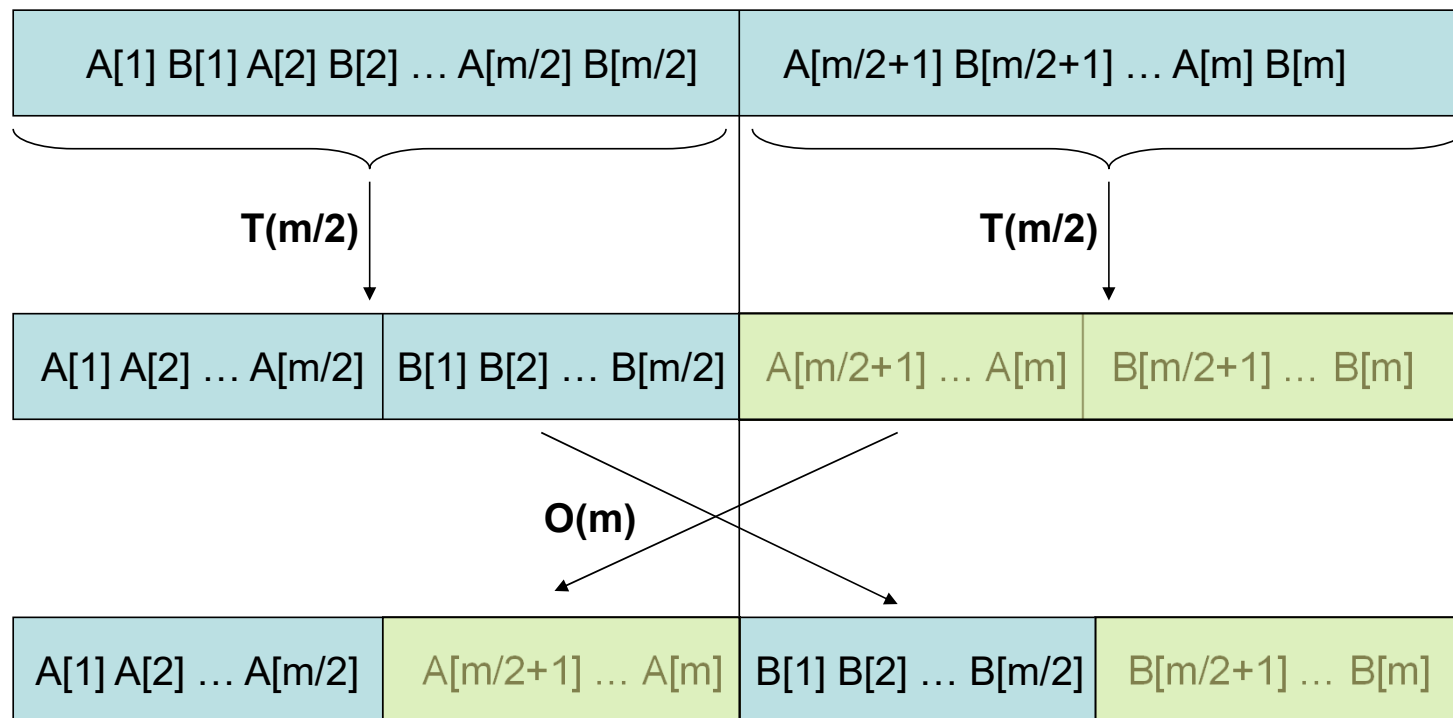
A[1] B[1] A[2] B[2] A[3] B[3] ... A[m] B[m]

↑ “Perfect shuffle”

A[1] A[2] A[3] ... A[m] B[1] B[2] B[3] ... B[m]



In-Place (Inverse) Perfect Shuffle In $O(m \log m)$ Time



General m x n Matrices

- Use in-place inverse perfect shuffle to build an $\Theta(N \log^2 N)$ divide-and-conquer algorithm for in-place matrix transposition!

